

Radix Base

Difficulty: Medium

Expect Time: 30 min

Author: Chen, YanHau

The conversion between number systems (base conversion) is a cornerstone of modern technology because it acts as the essential bridge between human logic and machine computation.

In this problem, you will be required to implement the `to_base<N>` method to convert a `DataRow` into a base-N string.

For example, `0x159486` this data can be converted into the following format:

- base-2: `101011001010010000110`
- base-10: `1414278`
- base-18: `D8910`
- base-36: `UB9I`

In `generate.h`, a `RandEngine` class has been implemented. Combined with the `gen::filter` method, it can generate `std::array<uint8_t, N>` data at compile time.

A 4-byte unsigned integer (e.g. `2393025178`) can be represented using `uint32_t` in hexadecimal

```
0x8ea2aa9a
```

Another way to represent it is by using a `uint8_t[4]` array:

```
{0x8e, 0xa2, 0xaa, 0x9a}
```

Example

The code below populates a `std::array` of length 1248 with random data. For this problem, note that the **array length is constrained to a multiple of 8**.

implement

```
template <int base, std::size_t N>
std::string to_base(std::array<uint8_t, N>& buf)
```

in `to_base.h`. This function should take a `std::array<uint8_t, N>` as input and return a base-N string, with no leading zeros.

For example, if `buf = 0x123456`, `to_base<10>(buf)` should return `"1193046"`, not `"01193046"` or `"0001193046"`; `buf = 0x0` should return `"0"`.

```
std::array<uint8_t, 1248> buf;
gen::fill(buf, 0x0000122b);
```

In this representation, the most significant byte(MSB) is stored at `buf[0]` , and the least significant byte(LSB) is stored at `buf[N-1]` .

Limits

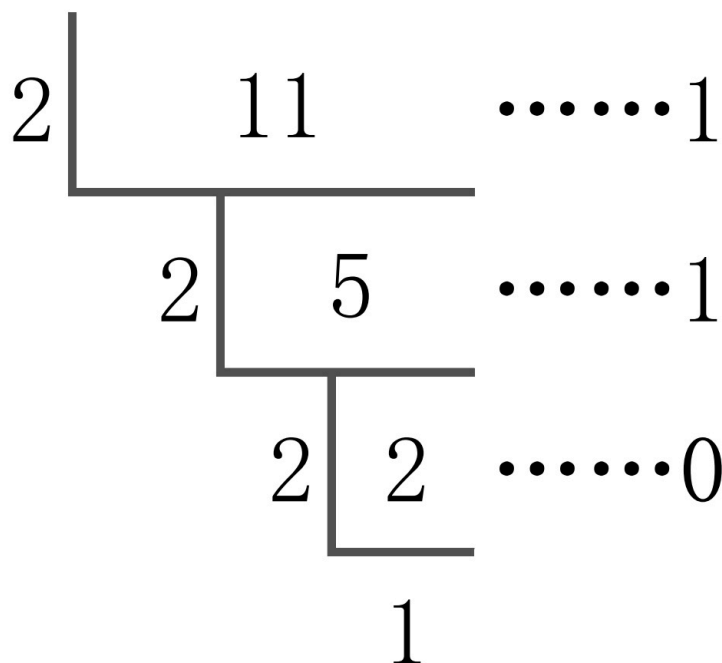
the Base range is 2 to 36, and `N` must be a multiple of 8 between `1` and `1500`, which corresponds to a positive integer in the range of $[0, 2^{12000})$.

For Base > 10, use uppercase for the output string; e.g., `to_base<16>({ 255 })` is expected to return 'FF', not 'ff'.

Hint

To convert a number from base-10 to any target base N using the repeated division method (often called the "short division" method), you essentially extract the remainders of the division process. This is the standard algorithm for base conversion. Here is the step-by-step logic:

- Divide: Take the number (the quotient) and divide it by the target base N .
- Record Remainder: Note down the remainder. This remainder represents the current digit in the new base (starting from the least significant digit).
- Update Quotient: Take the integer quotient from the previous division and use it as the new number to be divided.
- Repeat: Repeat steps 1–3 until the quotient becomes 0.
- Reverse: The final result in base N is obtained by reading the recorded remainders in reverse order (from the last one calculated to the first).



Input

1. Complete the required class implementation. The following files already contain some implementations:
 - `to_base.h`
 - `generate.h`
2. The Online Judge system will replace the following files. **DON'T** write implementation in those file:
 - `entrypoint.cpp`
 - `test.h`
 - `case.h`
3. The test cases for student available in `case*.h` .

Output

Online Judge will run the test:

1. **DON' T** output any information to the terminal.
2. When any test failed, Online Judge will try to list error messages.